

Generative Inference System Optimization

COMP4551

Binhang Yuan

Generative Inference

Recall Generative Inference Workflow

- State-of-the-art implementation splits the computation to two phrases:
 - Prefill phase: the model takes a prompt sequence as input and engages in the generation of a key-value cache (KV cache) for each Transformer layer.
 - Decode phase: for each decode step, the model updates the KV cache and reuses the KV to compute the output.
- Analyze the arithmetic intensity:
 - Prefill phrase: arithmetic bounded;
 - Decode phrase: memory bounded.
- Assume the computation is in fp16, the concrete analysis in next two slides.
 - L is the input sequence length;
 - D is the model dimension;
 - Multi-head attention $D = n_H \times H$; H is the head dimension; n_h is the number of heads.

Prefill Phrase



RELAXED
SYSTEM LAB

No.	Computation	Input	Output	Arithmetic Intensity
1	$Q = xW^Q$	$x \in \mathbb{R}^{L \times D}, W^Q \in \mathbb{R}^{D \times D}$	$Q \in \mathbb{R}^{L \times D}$	$\frac{L \times D^2}{L \times D + D^2 + L \times D} = \frac{L \times D}{2L + D}$
2	$K = xW^K$	$x \in \mathbb{R}^{L \times D}, W^K \in \mathbb{R}^{D \times D}$	$K \in \mathbb{R}^{L \times D}$	$\frac{L \times D^2}{L \times D + D^2 + L \times D} = \frac{L \times D}{2L + D}$
3	$V = xW^V$	$x \in \mathbb{R}^{L \times D}, W^V \in \mathbb{R}^{D \times D}$	$V \in \mathbb{R}^{L \times D}$	$\frac{L \times D^2}{L \times D + D^2 + L \times D} = \frac{L \times D}{2L + D}$
4	$[Q_1, Q_2 \dots, Q_{n_h}] = \text{Partition}_{-1}(Q)$	$Q \in \mathbb{R}^{L \times D}$	$Q_i \in \mathbb{R}^{L \times H}, i = 1, \dots, n_h$	-
5	$[K_1, K_2 \dots, K_{n_h}] = \text{Partition}_{-1}(K)$	$K \in \mathbb{R}^{L \times D}$	$K_i \in \mathbb{R}^{L \times H}, i = 1, \dots, n_h$	-
6	$[V_1, V_2 \dots, V_{n_h}] = \text{Partition}_{-1}(V)$	$V \in \mathbb{R}^{L \times D}$	$V_i \in \mathbb{R}^{L \times H}, i = 1, \dots, n_h$	-
7	$\text{Score}_i = \text{softmax}(\frac{Q_i K_i^T}{\sqrt{H}}), i = 1, \dots, n_h$	$Q_i, K_i \in \mathbb{R}^{L \times H}$	$\text{score}_i \in \mathbb{R}^{L \times L}$	$\frac{L^2 \times H}{L \times H + L^2 + L \times H} = \frac{L \times H}{2H + L}$
8	$Z_i = \text{score}_i V_i, i = 1, \dots, n_h$	$\text{score}_i \in \mathbb{R}^{L \times L}, V_i \in \mathbb{R}^{L \times H}$	$Z_i \in \mathbb{R}^{L \times H}$	$\frac{L^2 \times H}{L \times H + L^2 + L \times H} = \frac{L \times H}{2H + L}$
9	$Z = \text{Merge}_{-1}([Z_1, Z_2 \dots, Z_{n_h}])$	$Z_i \in \mathbb{R}^{L \times H}, i = 1, \dots, n_h$	$Z \in \mathbb{R}^{L \times D}$	-
10	$\text{Out} = ZW^O$	$Z \in \mathbb{R}^{L \times D}, W^O \in \mathbb{R}^{D \times D}$	$\text{Out} \in \mathbb{R}^{L \times D}$	$\frac{L \times D^2}{L \times D + D^2 + L \times D} = \frac{L \times D}{2L + D}$
11	$A = \text{Out} W^1$	$\text{Out} \in \mathbb{R}^{L \times D}, W^1 \in \mathbb{R}^{D \times 4D}$	$A \in \mathbb{R}^{L \times 4D}$	$\frac{4L \times D^2}{L \times D + 4D^2 + L \times 4D} = \frac{4L \times D}{5L + 4D}$
12	$A' = \text{relu}(A)$	$A \in \mathbb{R}^{L \times 4D}$	$A' \in \mathbb{R}^{L \times 4D}$	-
13	$x' = A'W^2$	$A' \in \mathbb{R}^{L \times 4D}, W^2 \in \mathbb{R}^{4D \times D}$	$x' \in \mathbb{R}^{L \times D}$	$\frac{4L \times D^2}{L \times D + 4D^2 + L \times 4D} = \frac{4L \times D}{5L + 4D}$

Decoding Phrase



No	Computationt	Input	Output	Arithmetic Intensity
1	$Q = Q_d = tW^Q$	$t \in \mathbb{R}^{1 \times D}, W^Q \in \mathbb{R}^{D \times D}$	$Q, Q_d \in \mathbb{R}^{1 \times D}$	$\frac{1 \times D^2}{1 \times D + D^2 + 1 \times D} = \frac{D}{2+D}$
2	$K_d = tW^K$	$t \in \mathbb{R}^{1 \times D}, W^K \in \mathbb{R}^{D \times D}$	$K_d \in \mathbb{R}^{1 \times D}$	$\frac{1 \times D^2}{1 \times D + D^2 + 1 \times D} = \frac{D}{2+D}$
3	$K = \text{concat}(K_{\text{cache}}, K_d)$	$K_{\text{cache}} \in \mathbb{R}^{L \times D}, K_d \in \mathbb{R}^{1 \times D}$	$K \in \mathbb{R}^{(L+1) \times D}$	-
4	$V_d = tW^V$	$t \in \mathbb{R}^{1 \times D}, W^V \in \mathbb{R}^{D \times D}$	$V_d \in \mathbb{R}^{1 \times D}$	$\frac{1 \times D^2}{1 \times D + D^2 + 1 \times D} = \frac{D}{2+D}$
5	$V = \text{concat}(V_{\text{cache}}, V_d)$	$V_{\text{cache}} \in \mathbb{R}^{L \times D}, V_d \in \mathbb{R}^{1 \times D}$	$V \in \mathbb{R}^{(L+1) \times D}$	-
6	$[Q_1, Q_2, \dots, Q_{n_h}] = \text{Partition}_{-1}(Q)$	$Q \in \mathbb{R}^{1 \times D}$	$Q_i \in \mathbb{R}^{1 \times H}, i = 1, \dots, n_h$	-
7	$[K_1, K_2, \dots, K_{n_h}] = \text{Partition}_{-1}(K)$	$K \in \mathbb{R}^{(L+1) \times D}$	$K_i \in \mathbb{R}^{(L+1) \times H}, i = 1, \dots, n_h$	-
8	$[V_1, V_2, \dots, V_{n_h}] = \text{Partition}_{-1}(V)$	$V \in \mathbb{R}^{(L+1) \times D}$	$V_i \in \mathbb{R}^{(L+1) \times H}, i = 1, \dots, n_h$	-
9	$\text{Score}_i = \text{softmax}(\frac{Q_i K_i^T}{\sqrt{H}}), i = 1, \dots, n_h$	$Q_i \in \mathbb{R}^{1 \times H}, K_i \in \mathbb{R}^{(L+1) \times H}$	$\text{score}_i \in \mathbb{R}^{1 \times (L+1)}$	$\frac{1 \times (L+1) \times H}{1 \times H + (L+1) \times H + L+1} = \frac{H+LH}{2H+L+LH+1}$
10	$Z_i = \text{score}_i V_i, i = 1, \dots, n_h$	$\text{score}_i \in \mathbb{R}^{1 \times (L+1)}, V_i \in \mathbb{R}^{(L+1) \times H}$	$Z_i \in \mathbb{R}^{1 \times H}$	$\frac{(L+1) \times H}{L+1 + (L+1) \times H + 1 \times H} = \frac{H+LH}{2H+L+LH+1}$
11	$Z = \text{Merge}_{-1}([Z_1, Z_2, \dots, Z_{n_h}])$	$Z_i \in \mathbb{R}^{1 \times H}, i = 1, \dots, n_h$	$Z \in \mathbb{R}^{1 \times D}$	-
12	$\text{Out} = ZW^O$	$Z \in \mathbb{R}^{1 \times D}, W^O \in \mathbb{R}^{D \times D}$	$\text{Out} \in \mathbb{R}^{1 \times D}$	$\frac{1 \times D^2}{1 \times D + D^2 + 1 \times D} = \frac{D}{2+D}$
13	$A = \text{Out} W^1$	$\text{Out} \in \mathbb{R}^{1 \times D}, W^1 \in \mathbb{R}^{D \times 4D}$	$A \in \mathbb{R}^{1 \times 4D}$	$\frac{1 \times 4D^2}{1 \times D + 4D^2 + 1 \times 4D} = \frac{4D}{5+4D}$
14	$A' = \text{relu}(A)$	$A \in \mathbb{R}^{1 \times 4D}$	$A' \in \mathbb{R}^{1 \times 4D}$	-
15	$t' = A'W^2$	$A' \in \mathbb{R}^{1 \times 4D}, W^2 \in \mathbb{R}^{4D \times D}$	$t' \in \mathbb{R}^{1 \times D}$	$\frac{1 \times 4D^2}{1 \times 4D + 4D^2 + 1 \times D} = \frac{4D}{5+4D}$

Algorithm Optimization (Last Time)

- Algorithm optimization:
 - “Slightly” change the original computation at its bottleneck to make it run much faster;
- Decrease the I/O volume:
 - Model compression, i.e., quantization;
 - Knowledge distillation.
 - KV cache optimization.

Inference System Optimization Objectives

Optimization Goal

- System optimization:
 - Improve the system efficiency without changing the original computation;
- What is the measurement for generative inference?
 - From the user's perspective, e.g.,:
 - First token generation latency;
 - Generation throughput;
 - From the system's perspective, e.g.,:
 - Serving throughput for each model replica;
 - SLO (service-level objective):
 - For example, 90% of the requests can be accomplished within 1.5X of the time when they are running alone on the same hardware.

LLM API

- LLM API is an important component for you to build your LLM applications.
- **Simplified Integration:** LLM APIs offer a straightforward way to incorporate language understanding into applications. Developers can send text inputs to the API and receive generated responses
- **Scalability:** Using APIs allows applications to scale their language processing capabilities based on demand.

```
python                                                                    Copy Edit

import openai

# Replace 'your-api-key' with your actual OpenAI API key
openai.api_key = 'your-api-key'

response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Can you explain how the OpenAI API works?"}
    ]
)

print(response['choices'][0]['message']['content'])
```

OpenAI API Call Python Example

API Leaderboard

		FEATURES ↗	MODEL INTELLIGENCE ↗	PRICE ↗	OUTPUT TOKENS/S ↗	LATENCY ↗	END-TO-END RESPONSE TIME ↗		
API PROVIDER ↕	MODEL ↕	CONTEXT WINDOW ↕	ARTIFICIAL ANALYSIS INTELLIGENCE INDEX ↕	BLENDED USD/1M Tokens ↕	MEDIAN Tokens/s ↕	MEDIAN First Chunk (s) ↕	TOTAL Response (s) ↕	REASONING Time (s) ↕	FURTHER ANALYSIS
 OpenAI	 o4-mini (high)	200k	70	\$1.93	126.6	36.00	39.94	N/A	Model Providers
 Microsoft Azure	 o4-mini (high)	200k	70	\$1.93	79.2	65.21	71.52	N/A	Model Providers
 Google	 Gemini 2.5 Pro	1m	68	\$3.44	214.6	29.55	31.88	N/A	Model Providers
 Microsoft Azure	 o3	128k	67	\$17.50	81.0	38.07	44.25	N/A	Model Providers
 xAI	 Grok 3 mini Reasoning (high)	131k	67	\$0.35	202.0	0.38	12.76	9.90	Model Providers
 xAI	 Grok 3 mini Reasoning (high) Fast	131k	67	\$1.45	214.9	0.46	12.10	9.31	Model Providers
 OpenAI	 o3-mini (high)	200k	66	\$1.93	156.9	46.97	50.16	N/A	Model Providers
 Microsoft Azure	 o3-mini (high)	200k	66	\$1.93	205.5	40.61	43.05	N/A	Model Providers
 OpenAI	 o3-mini	200k	63	\$1.93	156.3	14.34	17.54	N/A	Model Providers
 Microsoft Azure	 o3-mini	200k	63	\$1.93	201.4	12.64	15.13	N/A	Model Providers
 OpenAI	 o1	200k	62	\$26.25	68.9	50.87	58.13	N/A	Model Providers
 Microsoft Azure	 o1	200k	62	\$26.25	114.9	26.30	30.65	N/A	Model Providers
 Google	 Gemini 2.5 Flash (Reasoning) (AI_Studio)	1m	60	\$0.99	347.5	8.35	9.79	N/A	Model Providers
 Lambda	 DeepSeek R1	164k	60	\$0.95	38.2	0.55	75.05	61.41	Model Providers
 deepseek	 DeepSeek R1	64k	60	\$0.96	23.5	3.86	125.12	99.96	Model Providers
 Hyperbolic	 DeepSeek R1	128k	60	\$2.00	71.5	2.11	41.93	32.83	Model Providers
 AWS	 DeepSeek R1	128k	60	\$2.36	65.2	0.44	44.08	35.97	Model Providers

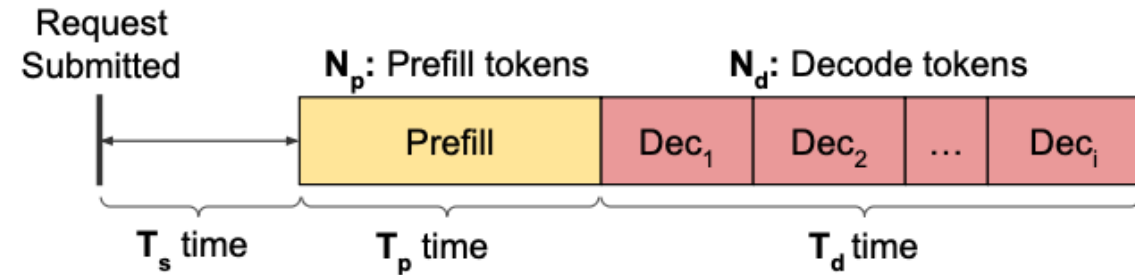
<https://artificialanalysis.ai/leaderboards/providers>

Evaluating LLM API Service

- When selecting a Large Language Model (LLM) API, besides output quality, it's crucial to consider various system-level factors:
- **Output speed** (tokens per second): This measures how quickly the model generates tokens. Higher output speeds are beneficial for applications requiring rapid responses.
- **Latency** (time to first token): Lower latency ensures faster initial responses, which is critical for real-time applications.
- **End-to-end response time**: This encompasses the total time from sending a request to receiving the complete response. It's a comprehensive metric for assessing user experience.
- **Pricing**: LLM APIs often charge based on the number of input and output tokens processed. It's essential to compare input and output token costs across providers.

Serving System Evaluation

- T_s : scheduling time;
- T_p : prefill processing time;
- T_d : decode generation time;



- **Time To First Token (TTFT)**: Latency between request arrival and first output token, including scheduling delay and prompt processing time ($T_s + T_p$). Critical for interactive applications requiring responsiveness.
- **Time to Last Token (TTLT)**: End-to-end latency ($T_s + T_p + T_d$) for complete request processing. Essential for applications like code completion where partial outputs have limited utility.
- **Time Between Tokens (TBT)**: Latency of each subsequent token generation (T_d^i) during decoding. Directly impacts perceived model speed, with 6 tokens/second matching typical reading speed.
- **Time Per Output Token (TPOT)**: Average decode time per token (T_d/N_d).

Optimization Overview

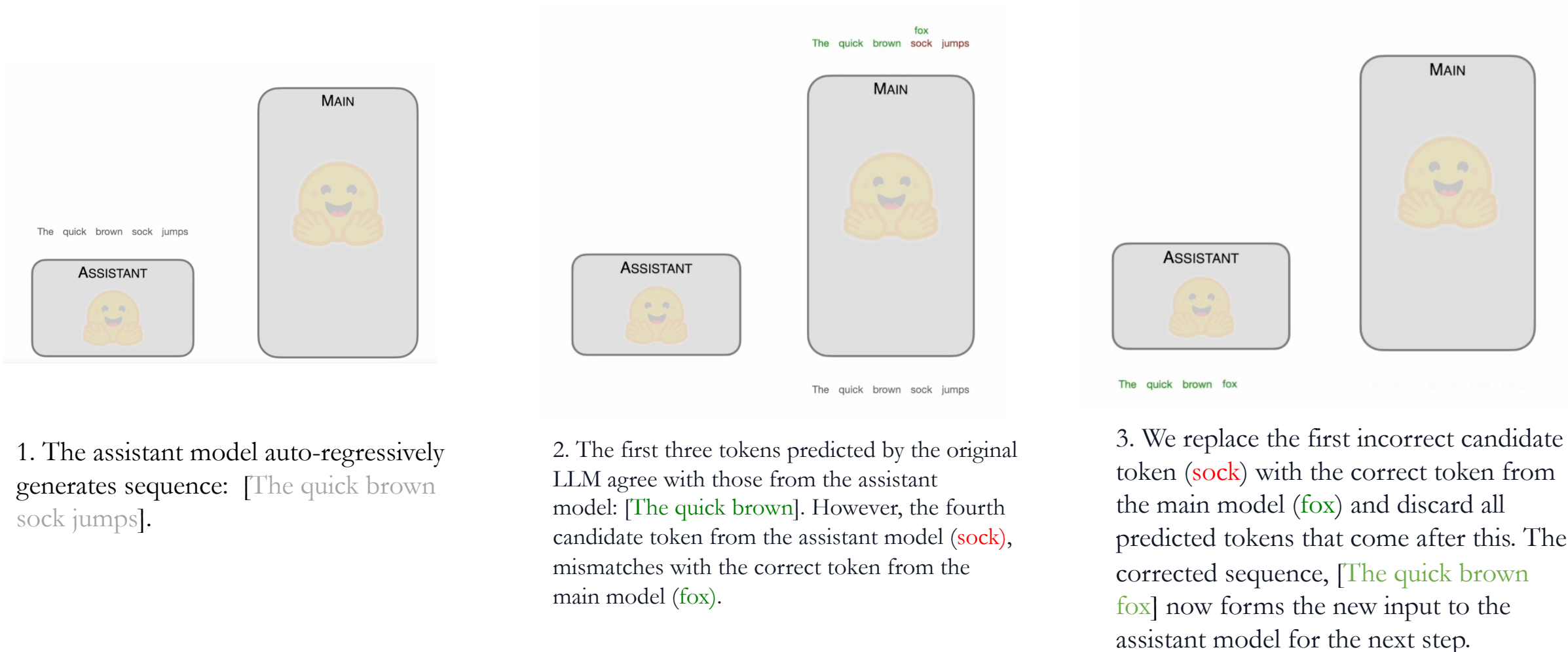
- System optimization:
 - Improve the system efficiency without changing the original computation;
- General ideas
 - For each request, generate multiple tokens simultaneously:
 - Speculative decoding;
 - Parallel decoding.
 - For multiple requests, effectively batching them:
 - Continuous batching;
 - Disaggregated inference.

Speculative Decoding

Speculative Decoding Overview

- Observation:
 - A small *assistant/speculative model* very often generates the same tokens as the large original LLM (sometime s referred to as the *main/target model*).
- Speculative decoding overview:
 - The assistant model auto-regressively generates a sequence of N candidate tokens;
 - The candidate tokens are passed to the original LLM to be verified. The original model takes the candidate tokens as input and performs *a single forward pass*:
 - All candidate tokens up to the first mismatch are correct;
 - After the first mismatch, replace the first incorrect candidate token with the correct token from the main model (fox), and discard all predicted tokens that come after this mismatched token.
 - Repeat this process until the end condition is reached.

Speculative Decoding Example



Speculative Decoding Notes

- We auto-regressively generate using the fast, assistant model, and only perform verification forward passes with the slow, main model, the decoding process is sped-up substantially.
- The verification forward passes performed by the main model ensures that exactly the same outputs are achieved as if we were using the main model standalone.
- Trade-off:
 - “The assistant model should be significantly faster.” V.S. “The assistant model should predict the same token distribution as often as possible.”
 - Since 70-80% of all predicted tokens tend to be "easier" tokens, this trade-off is heavily biased towards selecting a faster model, rather than a more accurate one.

Speculative Decoding in HF inference API

Example Code

```
from transformers import AutoModelForCausalLM, AutoTokenizer

prompt = "What is Apple?"

model_id = "EleutherAI/pythia-160m"
assistant_model_id = "EleutherAI/pythia-14m"

tokenizer = AutoTokenizer.from_pretrained(model_id)
inputs = tokenizer(prompt, return_tensors="pt")

model = AutoModelForCausalLM.from_pretrained(model_id)
assistant_model = AutoModelForCausalLM.from_pretrained(assistant_model_id)

outputs = model.generate(**inputs, max_new_tokens=20, assistant_model=assistant_model,
return_dict_in_generate=True)

input_length = inputs.input_ids.shape[1]
token = outputs.sequences[0, input_length+1:]
print(f"[INFO] raw token: {token}")
output = tokenizer.decode(token)
print(f"[Context]: {prompt} \n[Output]:{output}\n")
```

How to do sampling in Speculative Decoding? *

- We have two distribution now:
 - $x \sim p(x)$ sampling from the distribution of original model;
 - $x \sim q(x)$ sampling from the distribution of the assistant model;
- Speculative sampling:
 - To sample $x \sim p(x)$,
 - We instead sample from $x \sim q(x)$:
 - If $q(x) \leq p(x)$, keep x ;
 - Otherwise ($q(x) > p(x)$), reject the sample with probability $1 - \frac{p(x)}{q(x)}$ and sample x again from an adjusted distribution $p'(x)$.

$$p'(x) = \frac{p(x) - \min(q(x), p(x))}{\sum_{x'} (p(x') - \min(q(x'), p(x')))}$$

Fast Inference from Transformers via Speculative Decoding

Yaniv Leviathan^{*1} Matan Kalman^{*1} Yossi Matias¹

Abstract

Inference from large autoregressive models like Transformers is slow - decoding K tokens takes K serial runs of the model. In this work we introduce *speculative decoding* - an algorithm to sample from autoregressive models faster without any changes to the outputs, by computing several tokens in parallel. At the heart of our approach lie the observations that (1) hard language-modeling tasks often include easier subtasks that can be approximated well by more efficient models, and (2) using speculative execution and a novel sampling method, we can make exact decoding from the large models faster, by running them in parallel on the outputs of the approximation models, potentially generating several tokens concurrently, and without changing the distribution. Our method can accelerate existing off-the-shelf models without retraining or architecture changes. We demonstrate it on T5-XXL and show a 2X-3X acceleration compared to the standard TSX implementation, with identical outputs.

1. Introduction

Large autoregressive models, notably large Transformers (Vaswani et al., 2017), are much more capable than smaller models, as is evidenced countless times in recent years e.g., in the text or image domains, like GPT-3 (Brown et al., 2020), LaMDA (Thoppilan et al., 2022), Parti (Yu et al., 2022), and PaLM (Chowdhery et al., 2022). Unfortunately, a single decode step from these larger models is significantly slower than a step from their smaller counterparts, and making things worse, these steps are done serially - decoding K tokens takes K serial runs of the model.

Given the importance of large autoregressive models and specifically large Transformers, several approaches were

^{*}Equal contribution ¹Google Research, Mountain View, CA, USA. Correspondence to: Yaniv Leviathan <leviathan@google.com>.

Proceedings of the 40th International Conference on Machine Learning, Honolulu, Hawaii, USA. PMLR 202, 2023. Copyright 2023 by the author(s).

developed to make inference from them faster. Some approaches aim to reduce the inference cost for *all* inputs equally (e.g. Hinton et al., 2015; Jaszczur et al., 2021; Hubara et al., 2016; So et al., 2021; Shazeer, 2019). Other approaches stem from the observation that not all inference steps are born alike - some require a very large model, while others can be approximated well by more efficient models. These *adaptive computation* methods (e.g. Han et al., 2021; Sukhbaatar et al., 2019; Schuster et al., 2021; Scardapane et al., 2020; Bapna et al., 2020; Elbayad et al., 2019; Schwartz et al., 2020) aim to use less compute resources for easier inference steps. While many of these solutions have proven extremely effective in practice, they usually require changing the model architecture, changing the training-procedure and re-training the models, and don't maintain identical outputs.

The key observation above, that some inference steps are "harder" and some are "easier", is also a key motivator for our work. We additionally observe that inference from large models is often not bottlenecked on arithmetic operations, but rather on memory bandwidth and communication, so additional computation resources might be available. Therefore we suggest increasing concurrency as a complementary approach to using an adaptive amount of computation. Specifically, we are able to accelerate inference without changing the model architectures, without changing the training-procedures or needing to re-train the models, and without changing the model output distribution. This is accomplished via *speculative execution*.

Speculative execution (Burton, 1985; Hennessy & Patterson, 2012) is an optimization technique, common in processors, where a task is performed in parallel to verifying if it's actually needed - the payoff being increased concurrency. A well-known example of speculative execution is branch prediction. For speculative execution to be effective, we need an efficient mechanism to suggest tasks to execute that are likely to be needed. In this work, we generalize speculative execution to the stochastic setting - where a task *might be* needed with some probability. Applying this to decoding from autoregressive models like Transformers, we sample generations from more efficient *approximation models* as speculative prefixes for the slower *target models*. With a novel sampling method, *speculative sampling*, we maximize the probability of these speculative tasks to

arXiv:2211.17192v2 [cs.LG] 18 May 2023

<https://arxiv.org/pdf/2211.17192.pdf>

Speculative Decoding Optimization

- If the assistant model is asked to output the length m draft sequence, and the original LLM accepts n , $n < m$, the $(m - n)$ tokens are automatically discarded.
- If $n \ll m$, every LLM forward leads to only a limited number of tokens being decoded.
- Optimization, e.g., [SpecInfer <https://arxiv.org/pdf/2305.09781.pdf>]:
 - Let the assistant model(s) sample multiple plausible draft sequences for the original LLM to evaluate in parallel.
 - Sampling from multiple small assistant model drafts.
 - Organize the draft sequences as a tree to reuse the KV-cache. [Tree Attention]

Parallel Decoding

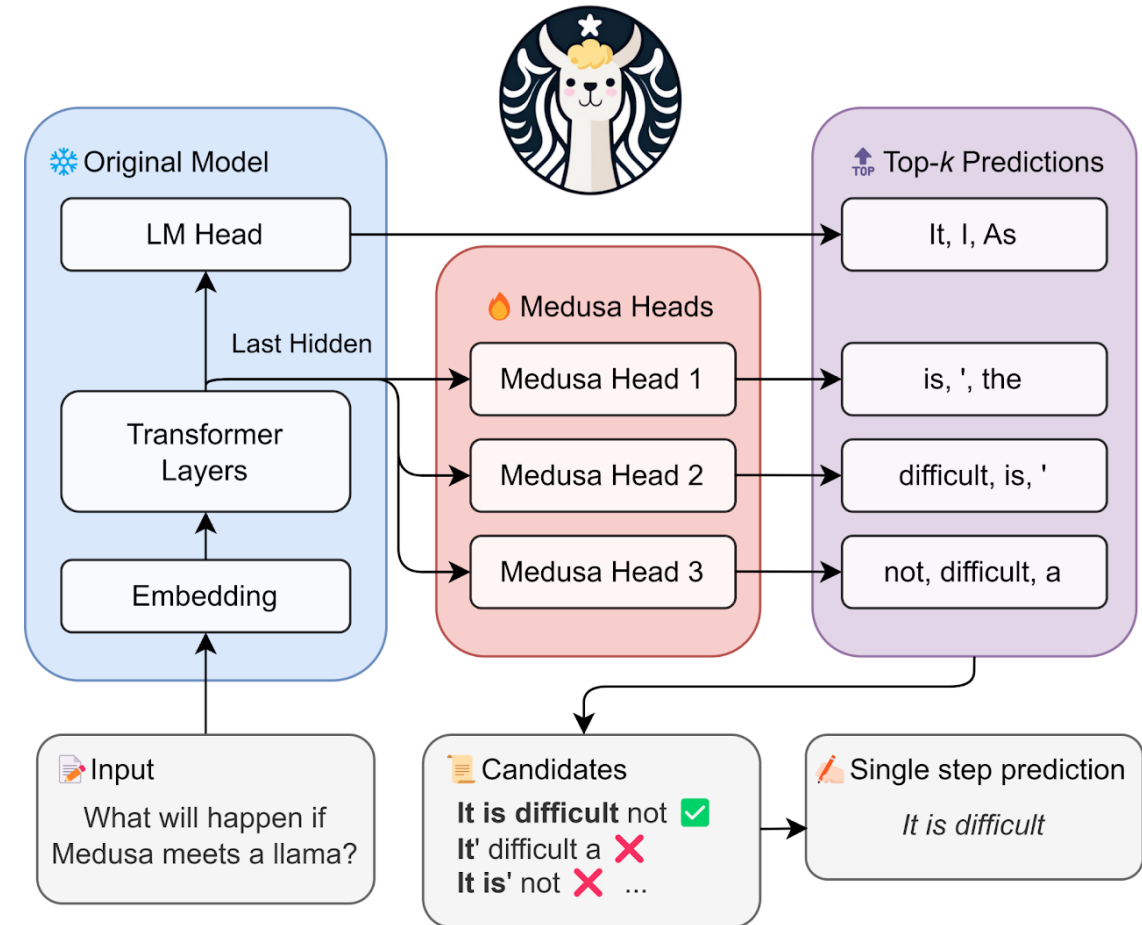
Parallel Decoding Overview

- Speculative decoding generates multiple token sequences using the assistant model for verification by the original LLM.
- Parallel decoding enables multiple token predictions directly from one forward pass of the original LLM.
- How to enable this?
 - [Medusa <https://arxiv.org/pdf/2401.10774.pdf>, <https://github.com/FasterDecoding/Medusa>]
 - Medusa heads (last layer projection);
 - Tree Attention.



Medusa Heads

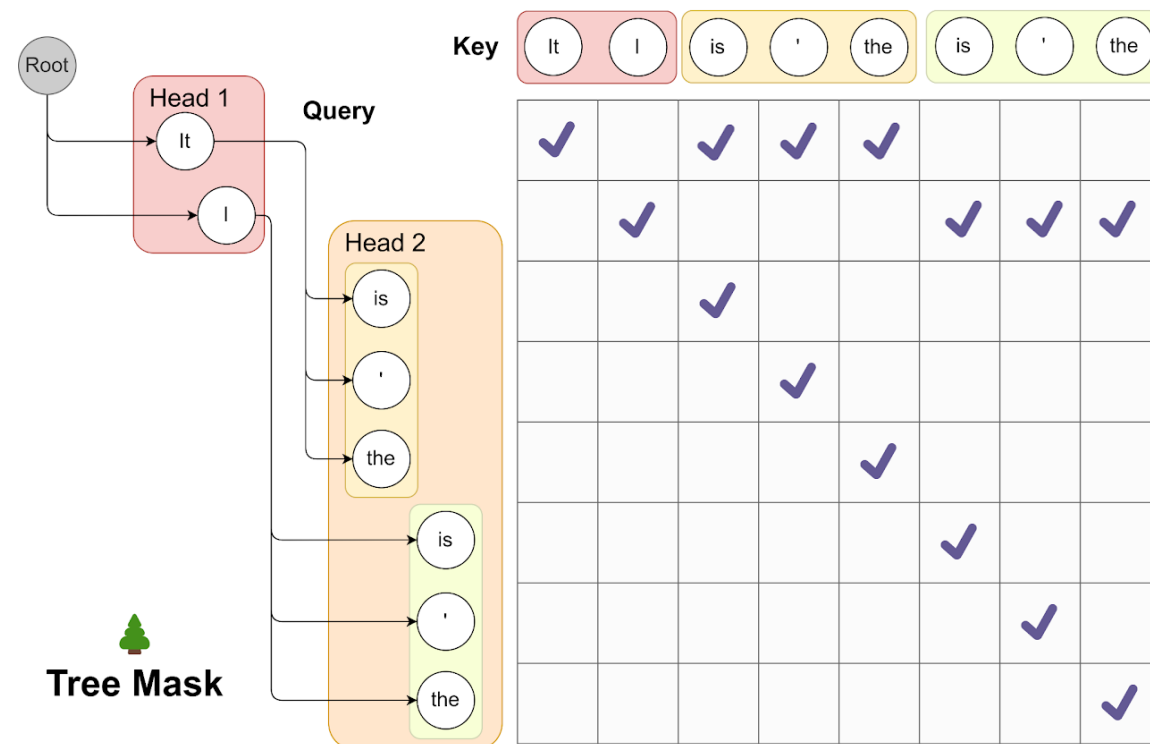
- Rather than pulling in an entirely new assistant model to predict subsequent tokens, Medusa simply extends the original LLM itself.
- Medusa heads are the additional decoding heads built on top of the last hidden states of the LLM, enabling the prediction of several subsequent tokens in parallel.
- Each Medusa head is a single layer of feed-forward network, augmented with a residual connection.
- Training these heads is straightforward: for a relatively small dataset, the original model remains static; only the Medusa heads are updated.



<https://www.together.ai/blog/medusa>

Tree Attention

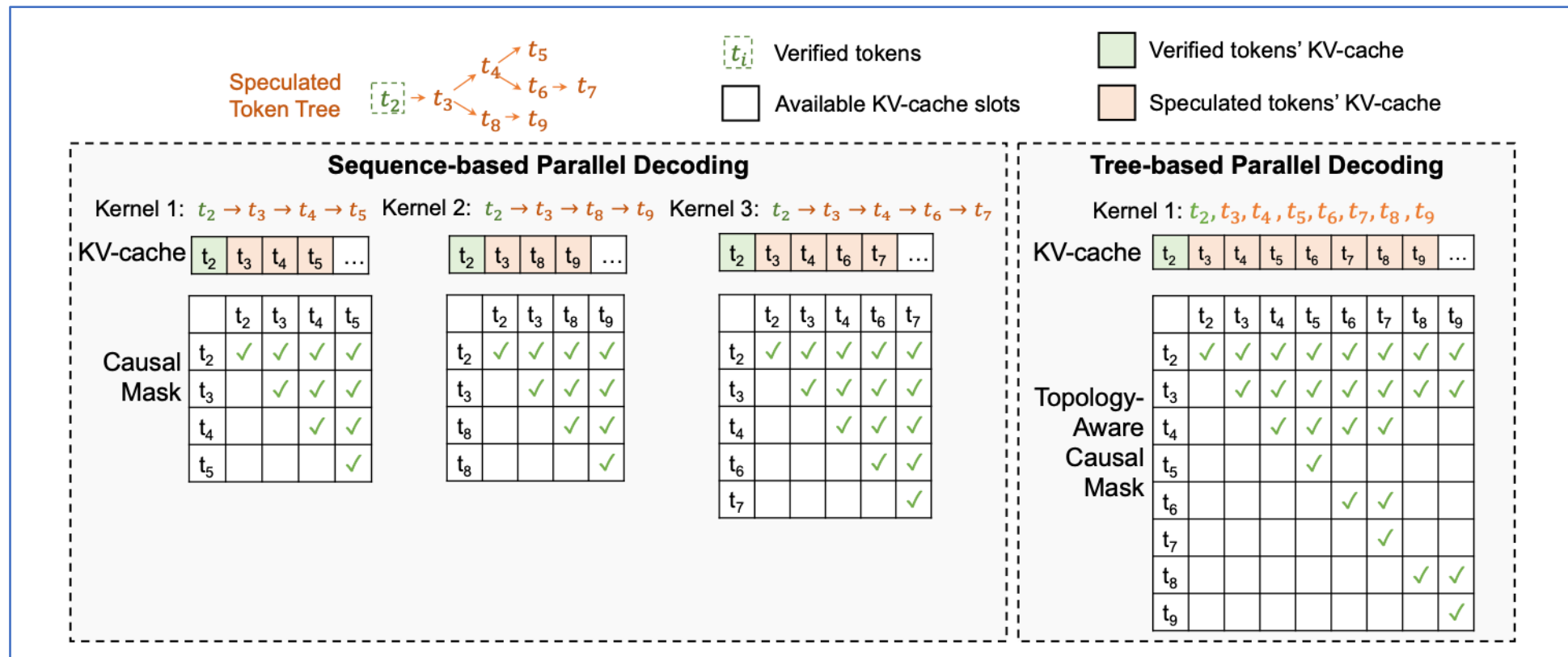
- Tree Attention in Medusa:
 - The top-1 accuracy for predicting the 'next-next' token hovers around 60%;
 - The top-5 accuracy soars to over 80%.
 - This substantial increase indicates that leveraging the multiple top-ranked predictions made by the Medusa heads can significantly amplify the number of tokens generated per decoding step.
 - Construct the Cartesian product of the top predictions from each Medusa head and encode the dependency graph into the attention.



Modified from <https://www.together.ai/blog/medusa> to avoid confusion when compared with SpecInfer

Tree Attention

- A similar idea can also be leveraged in speculative decoding when you organize multiple sequences as a prefix tree.



[Figure from SpecInfer.]

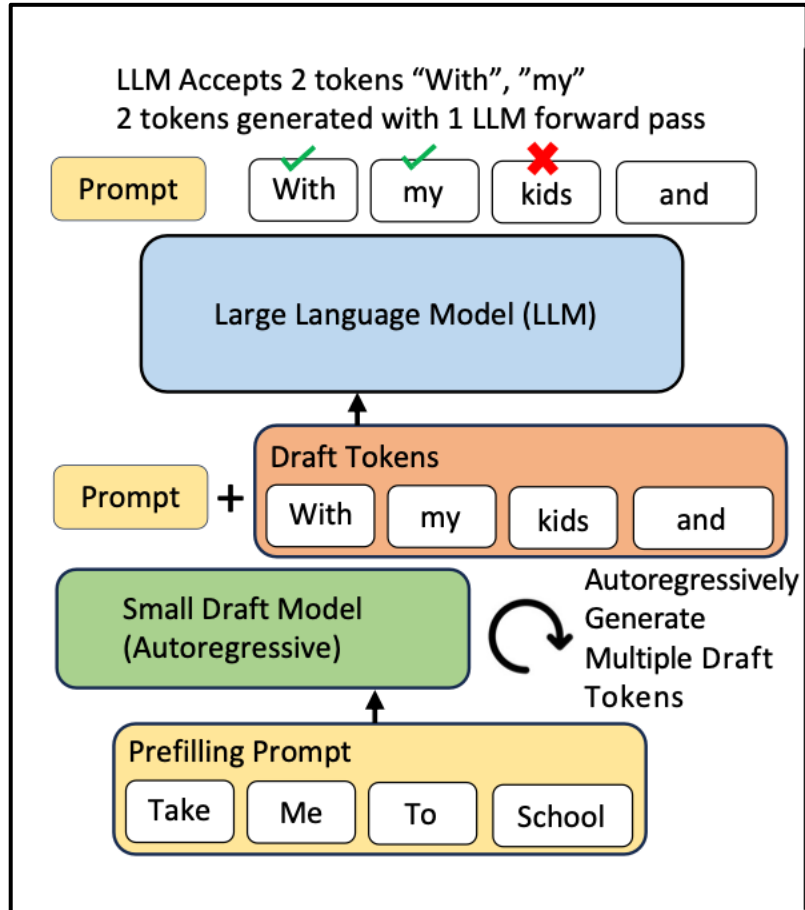
Prefix Tree

- A prefix tree, also named trie (pronounced “try”) is a tree-based data structure that stores strings/sequences efficiently by sharing common prefixes.
- A prefix tree enables fast string/sequence search, and insertion operations in $O(L)$ time, where L is the string length.
- In a prefix tree:
 - Each node represents a single character/token.
 - The path from the root to any node forms a prefix of one or more strings/sequences.
 - Strings/sequences that share prefixes use shared paths.

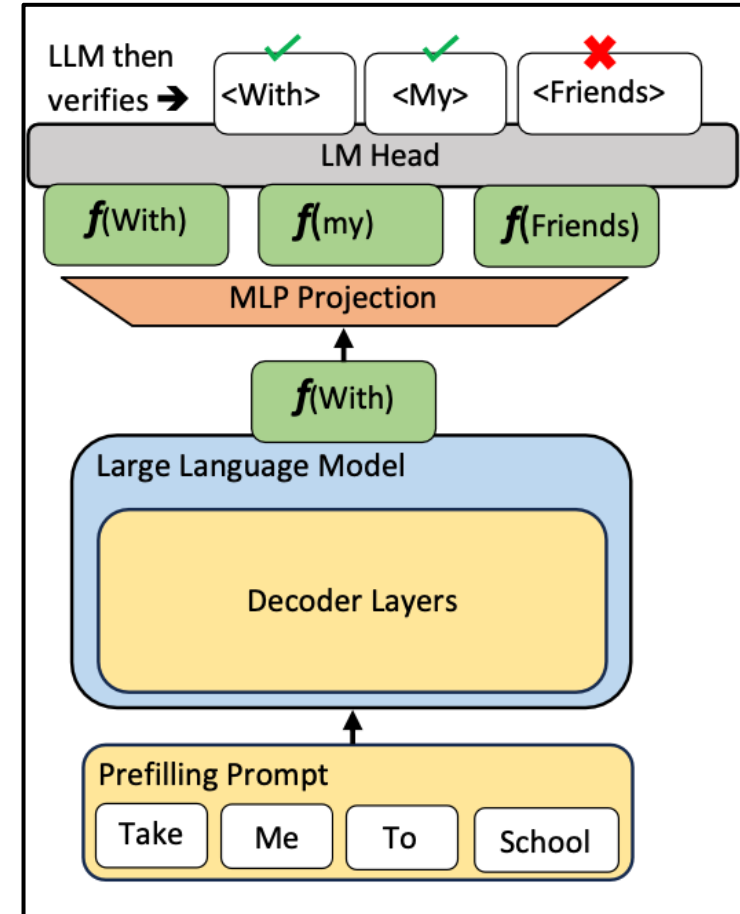
Prefix Tree

- **How to insert a sequence into a prefix tree:**
 - we start at the root and follow the tokens of the sequence one by one.
 - If a token path already exists, we move forward.
 - If not, we create a new node corresponding to that token. Once all tokens are inserted, it is done (we mark the end of the sequence.)
- **How to search a sequence in a prefix tree:**
 - When you want to check if a sequence exists in a prefix tree, follow its characters from the root.
 - The search is successful if all tokens match and the last one is marked as the end of a word. Otherwise, the sequence either doesn't exist or is just a prefix.

Speculative Decoding v.s. Parallel Decoding



Speculative Decoding



Parallel Decoding

Continuous Batching

Naive Batching

- The naïve approach is static batching:
 - The size of the batch remains constant until the inference is complete.
- Issue:
 - The inference process of an LLM is iterative.
 - Some requests may 'end' before the completion of the batch
 - This means that the GPU may be underutilized.

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3	S_3				
S_4	S_4	S_4	S_4	S_4			

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END		
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END			
S_4	S_4	S_4	S_4	S_4	S_4	END	

Yellow: Prefill
Blue: Decode

Continuous Batching

- Orca: <https://www.usenix.org/conference/osdi22/presentation/yu>
- Key idea:
 - Instead of waiting until every sequence in a batch has completed generation, Orca implements iteration-level scheduling;
 - Once a sequence in a batch has completed generation, a new sequence can be inserted in its place;
 - Higher GPU utilization than static batching.

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3					
S_4	S_4	S_4					

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

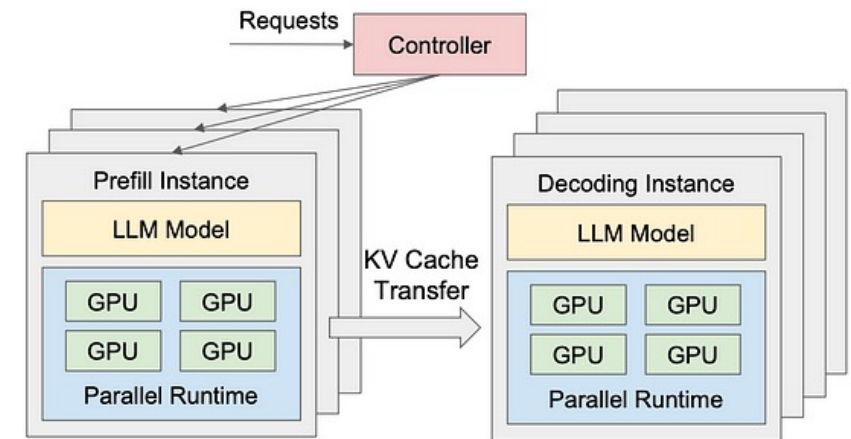
Disaggregated Inference

Disaggregated Inference

- Issues with batching based approach:
 - Prefill:
 - Computation-bounded $\Rightarrow$ Batching's benefit is marginal;
 - Decoding:
 - Memory-bounded $\Rightarrow$ Batching's benefit is significant.
 - Prefill-decoding interference:
 - You feel some sudden stall for some output token when you are using the streaming inference API.
 - Perhaps this is when a new request's prefill step is injected to your batch for decoding.

Disaggregated Inference

- DistServe:
<https://www.usenix.org/system/files/osdi24-zhong-yinmin.pdf>
- Key ideas:
 - Prefill computation on some GPUs;
 - Decoding computation on some other GPUs;
 - Prefill and decoding instances can have different parallel configurations;
 - Dynamic configuration of the prefill / decoding ratio;
 - Overhead: KV-cache communication.

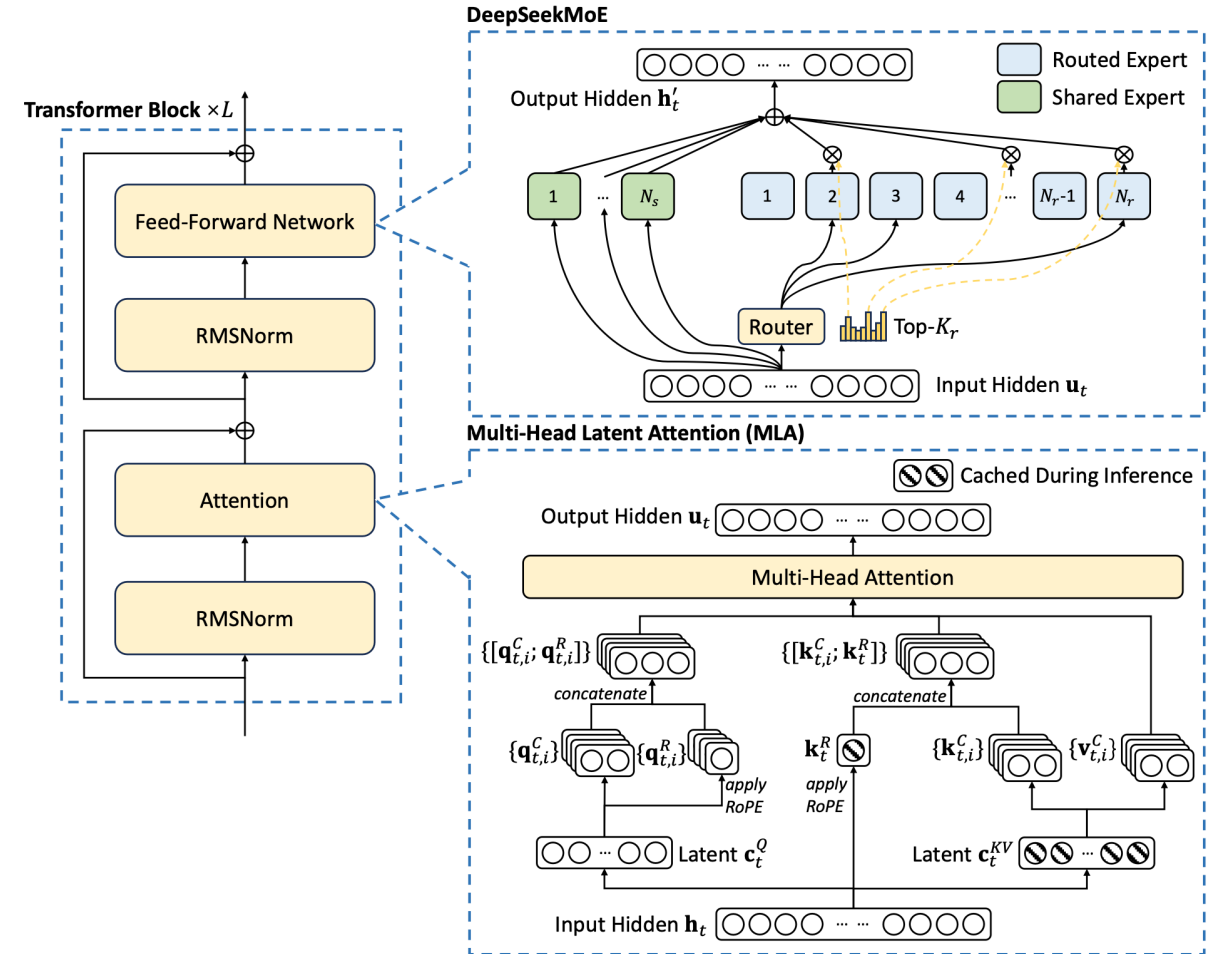


Deepseek Disaggregated Inference Service



Deepseek V3/R1 Model

- Multi-Head Latent Attention (MLA):
 - Similar memory footprint with MQA;
 - Similar performance with MHA.
- Deepseek-MoE:
 - Finely segmentation;
 - Shared expert



Deepseek V3/R1 Inference Service

- Minimum deployment unit of the prefilling stage configuration (4 node - 32 GPU):
 - MLA: 1-way TP & 32-way DP;
 - MoE: 32-way EP.
- Optimization:
 - Redundant MoE experts:
 - Duplicates high-load experts and deploys them redundantly;
 - The high-load experts are detected based on statistics collected during the online deployment and are adjusted periodically (e.g., every 10 minutes);
 - Set 32 redundant experts for the prefilling stage.
 - Rearrange MoE experts:
 - Rearrange experts among GPUs within a node based on the observed loads, striving to balance the load across GPUs as much as possible without increasing the cross-node all-to-all communication overhead.

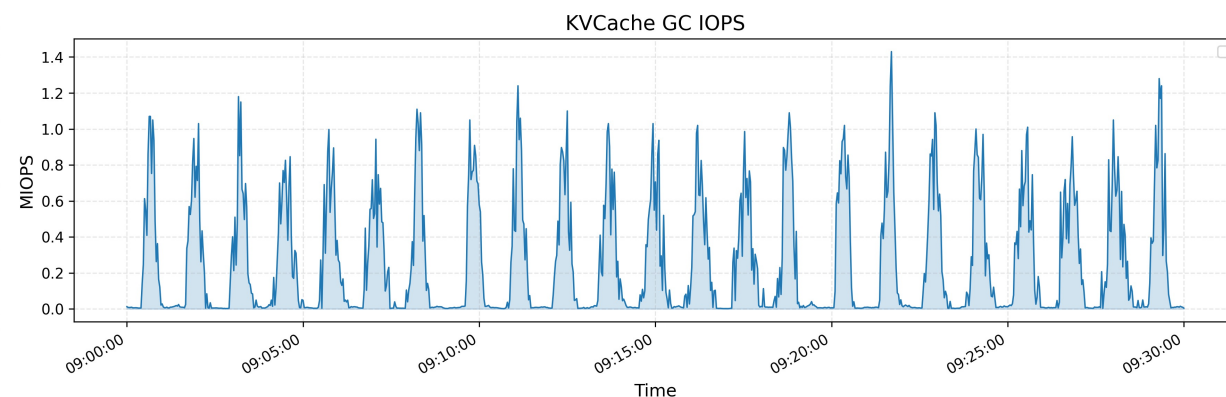
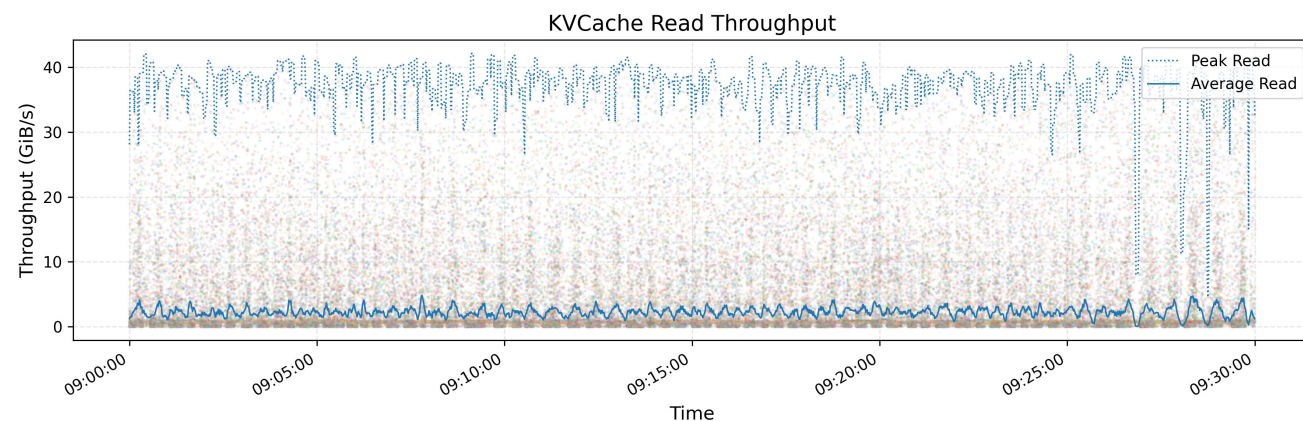
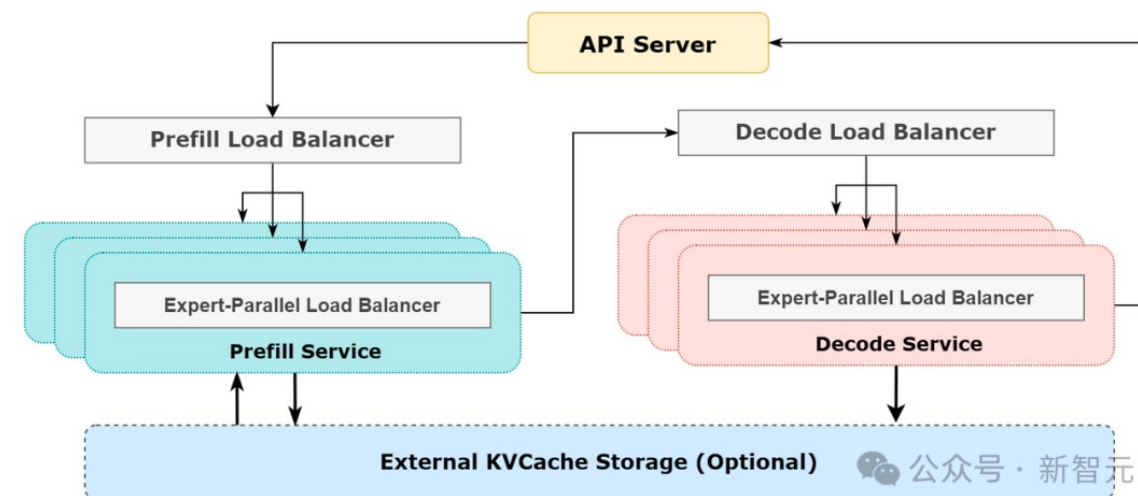
Deepseek V3/R1 Inference Service

- Minimum deployment unit of the decoding stage configuration (**18 node - 144 GPU**):
 - MLA: 1-way TP & 144-way DP;
 - MoE: 144-way EP (each GPU hosts only two routed experts).
- Try to optimize the batch size per expert (usually 256 tokens), but the bottleneck is memory access rather than computation.
- Optimization:
 - NVSHMEM are leveraged for All-to-all communication of the dispatch and combine part
 - 64 GPUs are responsible for hosting redundant experts and shared experts.



Deepseek V3/R1 Inference Service

- Efficient file system for KV-cache sharing.
- Read throughput of all KVCache clients ($1 \times 400\text{Gbps}$ NIC/node), highlighting both peak and average values, with peak throughput reaching up to 40 GiB/s.



<https://github.com/deepseek-ai/3FS>

References

- <https://huggingface.co/blog/whisper-speculative-decoding>
- <https://arxiv.org/pdf/2211.17192.pdf>
- <https://www.together.ai/blog/medusa>
- <https://arxiv.org/abs/2402.16363>
- <https://medium.com/@yohoso/llm-inference-optimisation-continuous-batching-2d66844c19e9>
- <https://www.usenix.org/conference/osdi22/presentation/yu>
- <https://www.usenix.org/system/files/osdi24-zhong-yinmin.pdf>
- <https://zhuanlan.zhihu.com/p/27181462601>
- <https://www.codecademy.com/article/trie-data-structure-complete-guide-to-prefix-trees>